

Proyecto de investigación: vulnerabilidad CVE-2021-3156

Kener Adiel Castillo
Ingeniería en Computación
Instituto Tecnológico de Costa Rica
Alajuela, Costa Rica
kencastillo@estudiantec.cr

Alejandro Cerdas Vargas
Ingeniería en Computación
Instituto Tecnológico de Costa Rica
Alajuela, Costa Rica
<https://orcid.org/0009-0002-0096-1824>

I. ABSTRACT

The vulnerability CVE-2021-3156, known as Baron Same-dit, is a heap buffer overflow in Sudo, one of the most widely used privilege management applications on Unix systems. This vulnerability, caused by an off-by-one error during argument processing in Shell mode, allows a local user to escalate privileges and gain root access. This paper aims to analyze the severity, technical operation, root weakness, pattern, and attack vector associated with this vulnerability. To this end, an analysis of the vulnerable code and the conditions necessary to exploit the flaw was performed. Subsequently, a laboratory environment was built using a virtual machine with a vulnerable Ubuntu 20.04, and the exploit was replicated using a PoC based on heap manipulation and the NSS module, achieving a successful privilege escalation. Furthermore, the impact of the mitigation measures proposed by the Sudo developers was verified by installing the patched version 1.9.5p2 and confirming that the vulnerability was no longer exploitable. Finally, the security implications of this type of vulnerability are discussed, along with the importance of code review and secure software design, as well as the need to apply best practices for administration and monitoring in critical systems.

II. INTRODUCCIÓN

Sudo es uno de los programas que suelen estar presentes en la mayoría de distribuciones de Unix y su principal función es realizar acciones con permisos de otros usuarios. A su vez, Unix es uno de los sistemas operativos más utilizados a nivel mundial. Tecnologías en el área médica, aviación y otras industrias usan sus diferentes distribuciones para su uso diario. Por lo tanto, un fallo en sudo puede significar en pérdidas, no solo monetarias, a nivel mundial. Trabajar en el ámbito de la ciberseguridad significa irse a dormir y, al despertar, descubrir que decenas de vulnerabilidades fueron explotadas en sistemas de todo el mundo. Eso mismo le ocurrió a Todd Miller con Common Vulnerability and Exposures (CVE)-2021-3156, conocida como Baron Samedit.

El objetivo de este trabajo de investigación es realizar un análisis de la severidad asociada a CVE-2021-3156, un estudio técnico de las razones por la que ocurre la vulnerabilidad, su debilidad, patrón y vector de ataque asociados. Adicional-

mente, se busca replicar la explotación de la vulnerabilidad y demostrar como se puede remediar la misma.

III. MARCO TEÓRICO

La mayoría de las vulnerabilidades que se descubren día a día son registradas en las bases de datos de MITRE. MITRE es una organización sin fines de lucro que provee grandes bases de datos para almacenar información de las vulnerabilidades de los sistemas del mundo. Inicialmente fue conocido como Semi-Automatic Ground Environment (SAGE) siendo el primer sistema de defensa aérea de Estados Unidos. MITRE ofrece 4 pilares fundamentales para estudiar las diferentes vulnerabilidades [1].

- MITRE ATT&CK
- CVE
- Common Weakness Enumeration (CWE)
- Common Attack Pattern Enumeration and Classification (CAPEC)

MITRE ATT&CK es una matriz de organización y clasificación de todas las tácticas y técnicas que los atacantes usan en escenarios reales [1]. Por otra parte, CVE es una lista de vulnerabilidades y exposiciones de seguridad divulgadas públicamente, cuyo destino es servir de línea base de la industria para dialogar en un entorno de vulnerabilidad determinada. La estructura del identificador único de CVE es CVE-año-NNNN, siendo NNNN el número de vulnerabilidad descubierta ese año. Además del identificador, el CVE presenta la descripción de la vulnerabilidad, versiones de software afectadas, soluciones o mitigaciones al fallo y referencias a recursos de la vulnerabilidad y su explotación [2], [3].

El CWE es un catálogo de debilidades y errores de software, cuyo principal objetivo es colaborar en la identificación, clasificación y mitigación de fallos comunes en el desarrollo de software. Su ID es de la forma CWE-NNNN donde NNNN es el número de debilidad. Adicionalmente presenta información como descripción, términos alternativos, consecuencias asociadas, mitigaciones y relaciones con otras debilidades [4], [5].

Finalmente CAPEC ofrece un catálogo de los ataques más comunes dirigidos a software. La idea es que los desarrolladores tengan la visión de un atacante e implementen las defensas necesarias. El ID de CAPEC es de la forma CAPEC-NNNN

donde NNNN es el número de patrón de ataque asociado. Cada CAPEC se compone de una descripción, probabilidad del ataque, severidad, relaciones con otros CAPEC, flujo de ejecución, prerequisites, habilidad necesaria y recursos requeridos [6], [7].

Fuertemente relacionado y avalado por MITRE, se encuentra Common Vulnerability Scoring System (CVSS), el cual es método estándar para medir la gravedad de las vulnerabilidades. A diferencia de CAPEC, CVE y CWE, CVSS es gestionado por FIRST. Mediante una calculadora, CVSS otorga una puntuación entre 0,0 y 10,0, utilizando una combinación de métricas. Estas puntuaciones ayudan a los equipos a priorizar y comunicar riesgos [8].

Para el correcto funcionamiento y mantener un sistema seguro es necesario un programa como Sudo. Formalmente, Sudo es una utilidad de los sistemas operativos que permite a los usuarios ejecutar programas con los privilegios de seguridad de otros usuarios de manera segura. Sudo nace en 1980 gracias a Bob Coggeshall y Cliff Spencer [9]. `/etc/sudoers` es un archivo que los administradores de Linux usan para asignar los derechos del sistema a usuarios del mismo [10]. Asimismo, Sudo posee varias opciones como lo son `-h` (muestra el menú de ayuda), `-k` (invalida marcas de tiempo), `-i` (simula un inicio de sesión real), `-e` (editar archivos con privilegios elevados), `-s` (inicia un shell con privilegios de root), `-l` (enumera los comandos permitidos para un usuario), entre otros [11]. El comando `sudoedit` es una extensión de `sudo -e` pero le permite al usuario elegir su editor de texto de preferencia [12].

Además, de sudo, muchos sistemas Unix tiene otro componente crucial, Name Service Switch (NSS) cuya función principal es permitirle al sistema buscar varios tipos de información como datos de usuarios, hostnames y servicios de red de diferentes fuentes, como archivos locales, DNS, entre otros. Básicamente, colabora en la carga de bibliotecas. Su principal archivo de configuración es `/etc/nsswitch.conf` y posee la función `__GI__nss_lookup_function` [13].

Un desbordamiento del búfer (**buffer overflow**) es una vulnerabilidad de software que puede ser usada por atacantes para manipular el comportamiento de programas o sistemas. Un búfer es una sección de memoria consecutiva temporal la cual se usa para almacenar información. Formalmente, un desbordamiento del búfer es la acción de un atacante que le permite alterar la ruta de ejecución y sobrescribe elementos en su memoria, lo que compromete la ruta de ejecución del programa [14], [15].

Entre las debilidades que se suele aprovechar para lograr un desbordamiento del búfer se encuentra off-by-one. Off-by-one es una debilidad muy común y consiste en tener un error de cálculo al momento de asignar un valor, excediéndose por 1 o quedando 1 por debajo del valor esperado. Especialmente tiene efecto, al momento de copiar o iterar en un string. Otra debilidad es conocida como improper null termination, que

ocurre producto de que no se termina correctamente o no se termina un string con el carácter nulo o algún equivalente. Esto último le genera la posibilidad al atacante de terminar el string como le convenga [16], [17].

Un familiar cercano del desbordamiento del búfer, es el heap overflow. Heap overflow es un desbordamiento del búfer, pero el segmento de memoria que se sobrescribe es la porción de memoria del heap, producto, por lo general, de que la asignación de memoria fue hecha con una rutina como `malloc()`. La porción heap normalmente contiene código que se ejecuta directamente por los sistemas operativos y aplicaciones para priorizar la ejecución de procesos de datos [18], [19]. Para mitigar los efectos de los desbordamientos de búfer, se implementó Address Space Layout Randomization (ASLR), la cual es una medida de seguridad para evitar el redireccionamiento de la ejecución de código a uno malicioso. ASLR ordena de forma aleatoria las direcciones de la información importante incluyendo las posiciones de la pila, heap y las bibliotecas [20].

Con el fin de revisar los sistemas en el mundo, se han creado herramientas a lo largo del tiempo para identificarlas antes que los atacantes. El problema en ciberseguridad, es que estas herramientas, son igual de útiles para los atacantes, como es el caso de fuzz. En la década de 1980, Barton Miller implementó fuzz. La idea de fuzz es producir intencionalmente entradas malformadas con el fin de encontrar vulnerabilidades. Se compone de un poeta, el mensajero y el oráculo. El poeta crea esos test cases maliciosos antes mencionados, el mensajero entrega esos test cases y el oráculo determina si el test generó una falla [21].

IV. EVALUACIÓN DE SEVERIDAD

Con la información antes recaudada, el grupo de investigación realizó el cálculo del puntaje CVSS, con el fin de tener una idea general de la vulnerabilidad que se pensaba replicar, así como, la dificultad que esta le presentaría al grupo. El grupo realizó un análisis de los datos que poseían y completó el formulario presentado en la calculadora, dando como resultado el string `CVSS:4.0/AV:L/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N` con un puntaje de 8.5 (*High*). Con el fin de verificar lo asertado que fueron los cálculos, se consultó la base de datos nacional de vulnerabilidades de National Institute of Standards and Technology (NIST), la cual al CVE-2021-3156 le asigna el string `CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H` con una puntuación de 7.8 (*High*). Normalmente, los CVSS presentados por NIST están en la versión 3.1, como es el caso. Por lo tanto se realizó una comparación entre los valores de ambos strings, tabla I.

Las métricas que coincidieron con el vector string oficial son las siguientes:

- Attack Vector (AV)
- Attack Complexity (AC)

Label	CVSS 3.1	CVSS 4
Attack Vector (AV)	Local (L)	Local (L)
Attack Complexity (AC)	Low (L)	Low (L)
Attack Requirements (AT)	–	None (N)
Privileges Required (PR)	Low (L)	None (N)
User Interaction (UI)	None (N)	None (N)
Confidentiality (3.1 C y 4.0 VC)	High (H)	High (H)
Integrity (3.1 I y 4.0 VI)	High (H)	High (H)
Availability (3.1 A y 4.0 VA)	High (H)	None (N)
Subsequent Confidentiality (SC)	–	None (N)
Subsequent Integrity (SI)	–	None (N)
Subsequent Availability (SA)	–	None (N)
Scope (S)	Unchanged (U)	–
Score	7.8 (High)	8.5 (High)

Cuadro I

COMPARACIÓN DEL STRING GENERADO POR LOS INVESTIGADORES CON LA CALCULADORA CVSS VERSIÓN 4.0 Y EL STRING OBTENIDO DEL NIST HECHO POR UNA CALCULADORA CVSS VERSIÓN 3.1 [22]

- User Interaction (UI)
- Confidentiality (C y VC)
- Integrity (I y VI)

Mientras que las métricas en las que se difirió son:

- Privileges Required (PR)
- Availability (A y VA)

Por otro lado, las métricas que difieren entre la versión 3.1 y 4 de CVSS son:

- Attack Requirements (AT, de la versión 4.0)
- Subsequent Confidentiality (SC, de la versión 4.0)
- Subsequent Integrity (SI, de la versión 4.0)
- Subsequent Availability (SA, de la versión 4.0)
- Scope (S, de la versión 3.1)

Como se puede notar, hay varias métricas que difieren entre las versiones 3.1 y 4.0 de CVSS. Principalmente, en la versión 4.0 se añade la categoría de métricas de impacto a sistemas subsiguientes (SC, SI y SA) que evalúan el impacto a la confidencialidad, integridad y disponibilidad de los sistemas adyacentes al sistema afectado, lo cual reemplaza a la métrica de alcance (S) de la versión 3.1. También se incorpora la métrica requerimientos de ataque (AT) para determinar si la realización del ataque depende de condiciones de ejecución o del ambiente de ejecución. En general, la versión 4.0 de CVSS es una extensión de la versión 3.1 para describir y evaluar de forma más precisa el impacto de un ataque a un sistema y sistemas subsiguientes.

Gracias a la comparación, los autores comprendieron que este ataque puede provocar una deshabilitación del servicio afectado por la vulnerabilidad, tal como lo señala NIST (métrica Availability), y se identificó que el atacante requiere ciertos privilegios, en particular ingresar al sistema (métrica Privileges Required). El CVSS presenta una herramienta con un gran aporte a nivel informativo con el fin de que los usuarios de todo el mundo evalúen y revisen el impacto de los ataques informáticos a los sistemas vulnerables. Sin embargo, a nivel empresarial, no parece ser la mejor forma de medir una vulnerabilidad. A pesar de que la versión 4.0 presenta más especificaciones, los ataques de tipo ingeniería social

```
if (ISSET(sudo_mode, MODE_SHELL|MODE_LOGIN_SHELL)) {
    for (to = user_args, av = NewArgv + 1; (from = *av); av++) {
        while (*from) {
            if (from[0] == '\\' && !isspace((unsigned char)from[1]))
                from++;
            *to++ = *from++;
        }
        *to++ = ' ';
    }
}
```

Figura 1. Fragmento de código vulnerable a heap-based buffer overflow en el archivo `sudoers_policy_main()`

siguen sin poder otorgar un peso adicional al puntaje, así como las vulnerabilidades relacionadas con el hardware y dispositivos adyacentes, aparte de computadoras. Adicionalmente, las métricas pueden resultar muy generales o pueden depender de la interpretación, por lo tanto, ciertas compañías pueden menospreciar vulnerabilidades con baja puntuación que fueron catalogadas de forma subjetiva. En síntesis, se considera una herramienta con un gran valor para dar a conocer una severidad aproximada de las vulnerabilidades, mas no, debería ser el criterio de decisión dentro de una empresa.

V. ANÁLISIS DE LA VULNERABILIDAD

V-A. Contexto Histórico

La vulnerabilidad analizada nace en julio de 2011 en el commit 8255ed69 de sudo. Desde entonces se afectó a todas las versiones desde la 1.8.2 a la 1.8.31p2 y las versiones 1.9.0 a 1.9.5p1 en sus configuraciones estándares. Fue descubierta el 13 de enero del 2021 por Qualys Security Advisory y fue resuelta el 26 de enero del 2021 (13 días después), con los commits 1f86385 y b301b46. No existe registro de que la vulnerabilidad haya sido explotada activamente [23]–[25]. El método que usaron para descubrir la vulnerabilidad fue un análisis estático del código.

V-B. Descripción Técnica

Baron Samedit es una vulnerabilidad de desbordamiento de búfer en memoria heap (heap-based buffer overflow) que afecta al sistema de administración de usuarios Sudo, causada por un error de tipo off-by-one durante el procesamiento de argumentos de línea de comandos. Esta vulnerabilidad permite el escalamiento de privilegios para obtener privilegios de usuario raíz (root user) del sistema y es explotable por cualquier usuario local sin necesidad de autenticarse (no se necesita saber la contraseña del usuario).

Específicamente, si Sudo es ejecutado para correr un comando en Shell-mode una parte del programa concatena todos los parámetros de la línea de comandos y escapa todos los metacaracteres con barras invertidas (\). Sin embargo, más adelante el programa hace el proceso contrario (unscapes), asumiendo que después de cada barra invertida existe un carácter [24].

Como se observa en la figura 1, si `from[0]` es el caracter barra invertida y `from[1]` no es un espacio, `from[0]` se omite y `from[1]` se copia al buffer. Si un argumento de la línea de comandos terminara con una sola barra invertida, `from[1]` sería el delimitador del final del string, este se copiaría al buffer `user_args` y se incrementaría el contador. Lo que significa que se copiarían caracteres fuera de rango, pudiendo sobrepasar el tamaño del búfer y ocasionando un desbordamiento del heap, ya que este búfer se encuentra en el heap.

No obstante, en teoría, ningún argumento puede terminar en barra invertida ya que antes de llegar al código vulnerable se activa el analizador sintáctico (parser) debido a ciertas banderas activadas, el cual escapa caracteres especiales como barra invertida. Resulta ser, que al usar el comando `sudoedit -s` se activan diferentes banderas y se puede alcanzar el código vulnerable sin escapar caracteres. De esta forma, se puede producir un desbordamiento del búfer controlando los datos que se sobrescriben en el heap e insertando caracteres nulos. Con la técnica adecuada se podría manipular el programa afectado para que ejecute código o cargue alguna biblioteca maliciosa que aproveche los privilegios de Sudo para infiltrar el sistema.

V-C. Debilidad raíz

Según se observa en la página oficial de CVE el CWE relacionado a esta vulnerabilidad es el número 193 Off-by-One Error, lo cual se relaciona también con 170 Improper Null termination. Esta debilidad permite saltarse el delimitador que representa el final del string y sumado a un mal cálculo del tamaño de la entrada, ocasiona la posibilidad de hacer un desbordamiento del heap donde se ejecuta el programa. Específicamente, la función `set_cmds()` es vulnerable debido a que se puede inducir a un estado de “out of bound”, gracias a esta debilidad.

Adicionalmente, permite la vulnerabilidad de Sudo debido a que este es un programa que corre con privilegios de administrador aunque lo invoque un usuario normal. De esta forma un atacante, por medio de un desbordamiento de heap o buffer, puede lograr corromper la memoria y comportamiento de ese proceso privilegiado, logrando alterar el flujo de ejecución o estado interno y conseguir ejecución de código arbitrario con privilegios de administrador. En otras palabras, el atacante puede aprovechar el desbordamiento de memoria para modificar el estado del programa y poder ejecutar un script que le brinde privilegios de administrador en base a los privilegios de Sudo.

V-D. Patrón y vector de ataque

Se determinó que el CAPEC que aplica es el número 100 Overflow Buffers, ya que, con base en lo expuesto anteriormente, es posible aprovechar la debilidad mediante el desbordamiento del heap para vulnerar Sudo. CAPEC-100 es bastante común y tiene un impacto muy alto, además, el procedimiento para desbordar es catalogado que requiere un bajo nivel de habilidad, sin embargo, la habilidad necesaria

```
for (av = argv; *av != NULL; av++) {
    for (src = *av; *src != '\0'; src++) {
        if (!isalnum((unsigned char)*src) && *src != '_' && *src != '-' && *src != '$')
            *dst++ = '\\';
        *dst++ = *src;
    }
    *dst++ = ' ';
}
```

Figura 2. Fragmento de código de la función `parse_args()`

```
if (ISSET(mode, MODE_RUN) && ISSET(flags, MODE_SHELL))
```

Figura 3. Fragmento de código de la función `parse_args()`

para redigir el flujo del programa es catalogado como alta [26]. Esto nos deja con un vector de ataque ideal para una persona maliciosa. El vector de ataque se basa en su totalidad en si es posible acceder al código vulnerable sin ingresar a la función de `parse_args()`, es decir, es posible acceder a la función `set_cmd()` con la bandera `MODE_SHELL` y `MODE_EDIT` o `MODE_CHECK`, sin tener la bandera `MODE_RUN` para ocasionar un desbordamiento del heap [24], todo esto por comandos de terminal sin contar con privilegios.

El motivo por el que esta sería la entrada o vector de ataque, es que si no se cumple la condición mostrada en la figura 3 no se llegará a ejecutar la función de `parse_args()`, figura 2, la cual procesa la entrada y escapa los metacarácteres. Si se logra evadir esta función y si se cumple la condición de la figura 4, se podría realizar un ataque como se explica en la sección V-B [24].

VI. REPLICACIÓN DE LA VULNERABILIDAD

VI-A. Entorno de laboratorio

Para la replicación de la vulnerabilidad, primero fue necesario realizar un entorno de laboratorio. Primero se monto una máquina virtual de **Ubuntu 20.4 LTF Focal Fossa Beta** sobre el software de **Oracle VM Virtualbox** versión 7.0.16_Ubuntu. La versión de Ubuntu seleccionada presenta sudo 1.8.31. La máquina virtual fue configurada con 2048MB de memoria base, 1 procesador, sin ningun límite de ejecución, 16MB de memoria de video y la configuración base de la memoria y red. Una vez descargada la ISO en la Virtualbox, no fue necesario realizar ninguna configuración adicional. En la figura 5, se muestra como se obtuvo una configuración exitosa sin privelegio alguno. El `sudo nano` realiza la ejecución de nano con permisos de sudo. El motivo por el que se ejecuta es verificar que el usuario `vboxuser` no tiene privilegios de sudo. El comando `cat /etc/sudoers` muestra el contenido del archivo `/etc/sudoers` en terminal. Como se puede observar en la figura 5, ejecutar `cat` sobre este archivo requiere ciertos privilegios.

```
if (sudo_mode & (MODE_RUN | MODE_EDIT | MODE_CHECK))
```

Figura 4. Fragmento de código de la función `set_cmd()`

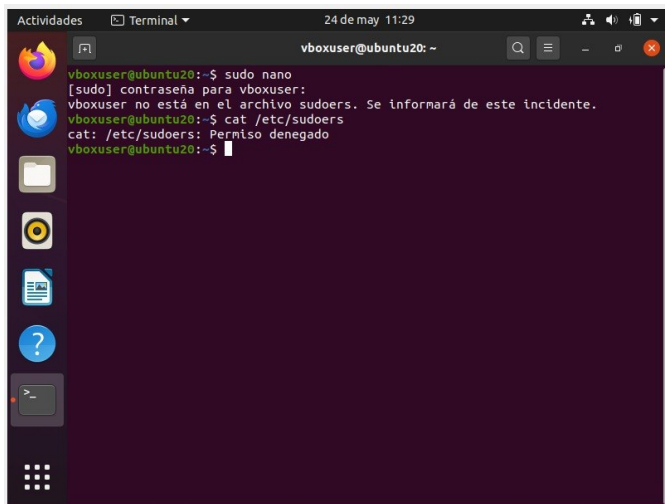


Figura 5. Ejecución de comandos sudo nano y cat /etc/sudoers

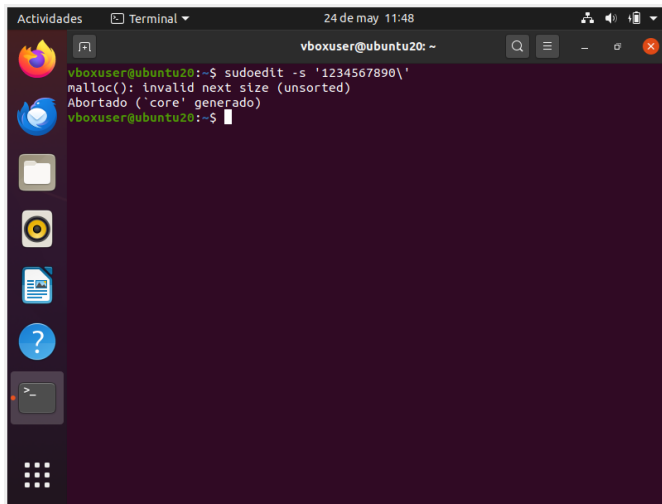


Figura 6. Demostración del Heap Overflow producto de CVE-2021-3156 con el comando `sudoedit -s '1234567890\'`

VI-B. Herramientas para la explotación

Causar el desbordamiento del heap es relativamente directo con Sudoedit, lo complicado es lograr manipular el espacio de memoria del heap en donde ocurre el desbordamiento para hacer que el programa vulnerado ejecute o llame a cierto script para atacar el sistema aprovechando los privilegios de administrador de Sudo. Respecto a esta vulnerabilidad, existen muchos Proof of Concept (PoC) que se basan en la manipulación de la función `__GI___nss_lookup_function` del módulo NSS para forzar que cargue y ejecute una biblioteca maliciosa por medio de la definición de variables de entorno locales y argumentos de tamaño aleatorio que logran una cierta distribución del heap.

Para este trabajo se hizo uso de los scripts que se encuentran en el repositorio Pwnedit para obtener privilegios de administrador por medio de la vulnerabilidad descrita. Este proceso consiste en la ejecución del comando `sudoedit -s` con ciertos argumentos, obtenidos mediante un proceso de fuzzing y análisis del heap, que ocasionan un desbordamiento del heap y una distribución específica del heap para manipular el comportamiento del módulo NSS. El objetivo es hacer que se cargue y ejecute una biblioteca definida, la cual asigna privilegios de administrador al usuario que está realizando el ataque. La razón de la elección de esta herramienta es que es mucho más simple y directa comparada con otros PoC. La desventaja es que solo funciona en una versión específica de Linux, pero esto permite que se puedan usar argumentos predefinidos en comparación con otros métodos aleatorios de prueba y error. Por otro lado, este método resultó funcionar la mayoría de las veces en que se probó.

VI-C. Explotación

Una vez configurado el ambiente y un usuario sin privilegios, como primer paso es verificar la existencia de la vulnerabilidad. En la figura 6 se ejecuta el comando `sudoedit -s 'DATA\'`. Sudoedit es un modo parte del

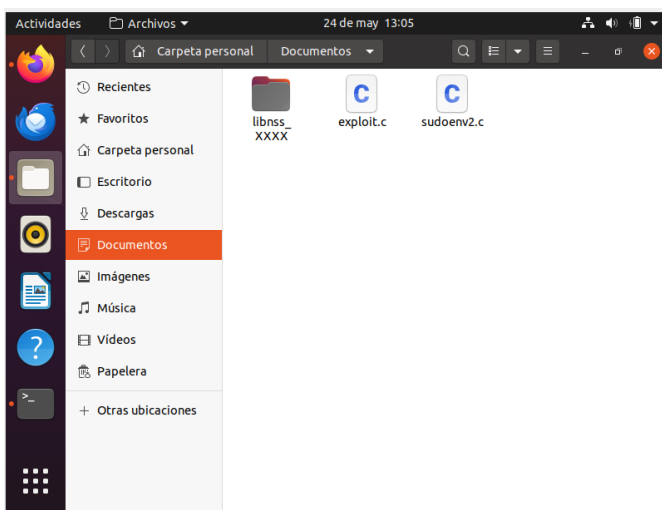


Figura 7. Archivos tomados del repositorio donde `exploit.c` es el archivo llamado `asd6.py` en el github

programa Sudo para editar archivos de forma más segura y la opción `-s` activa el modo Shell, esto permite entrar en ruta especial de procesamiento de argumentos, la cual maneja barras invertidas y escapes de forma incorrecta como se explica en la sección V-B. La barra invertida al final del argumento ocasiona un desbordamiento del heap, lo que resulta en el error que se muestra relacionado con malloc.

Del repositorio Pwnedit usado como herramienta, se sacan los archivos necesarios que se muestran en la figura 7.

En la carpeta `libnss_XXXX` se encuentra el archivo `liveoverflow.c`, como se ve en la figura 8, el cual es la biblioteca que se busca ejecutar por medio del desbordamiento del heap.

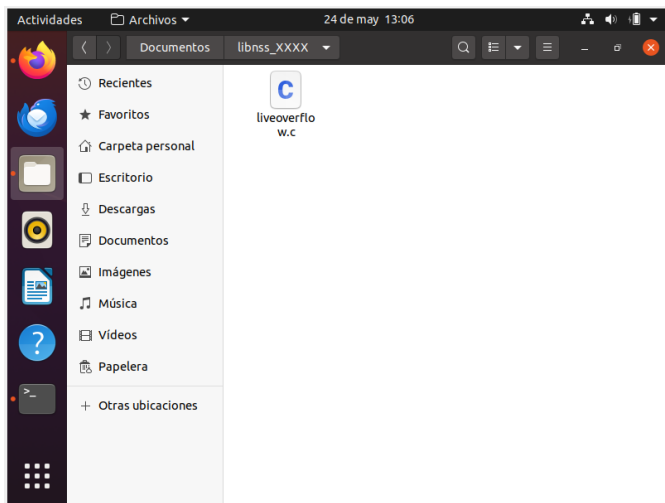


Figura 8. Contenido de la carpeta libnss_XXXX

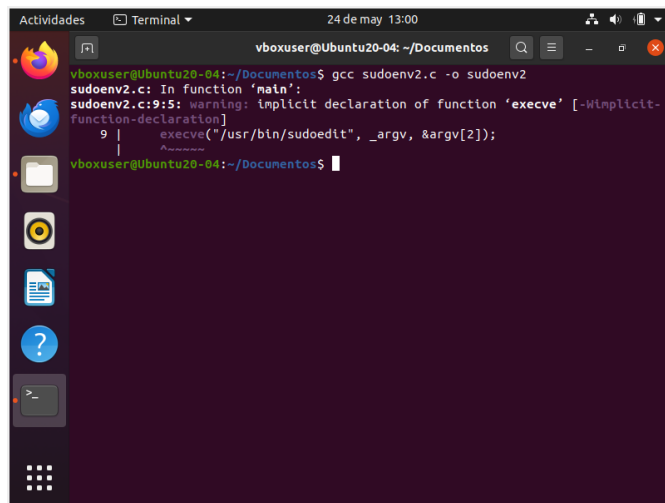


Figura 10. Comando para ejecutar archivo sudoenv2.c

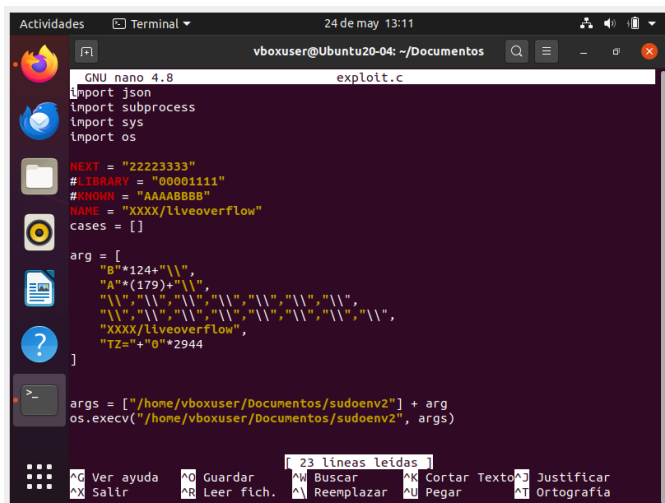


Figura 9. Cambios necesarios en el archivo exploit.c en la ruta del archivo sudoenv2

En la figura 9 se observa el archivo de Python usado para realizar el ataque. En este se definen en `arg` distintos strings que corresponden a argumentos para ejecutar el programa `sudoenv2`. El primer string corresponde al argumento para ejecutar `sudoedit -s` y el resto a variables de entorno.

En la figura 10 se compila el archivo de C `sudoenv2` con el comando `gcc Fuente -o Destino`, el resultado es ejecutado por `exploit.c`. Por otro lado, en la figura 11 se muestra el contenido del archivo. En este se ejecuta el comando `sudoedit -s` con `argv[1]` como argumento, el cual contiene barra invertida al final, y el resto de argumentos se establecen como variables de entorno para Sudoedit (se pasa `&argv[2]`).

A pesar de la aleatoriedad del heap que introduce métodos de seguridad como ASLR, no se destruye el orden relativo

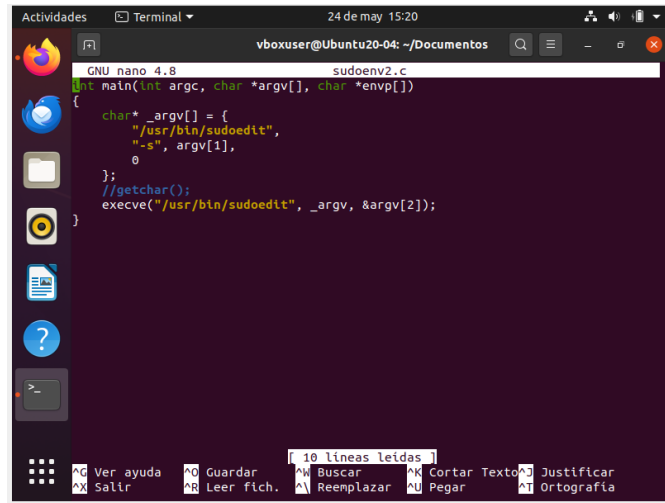
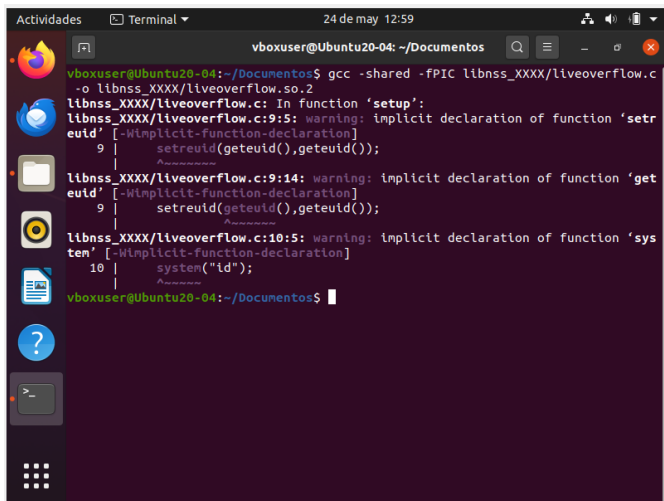


Figura 11. Contenido del archivo sudoenv2

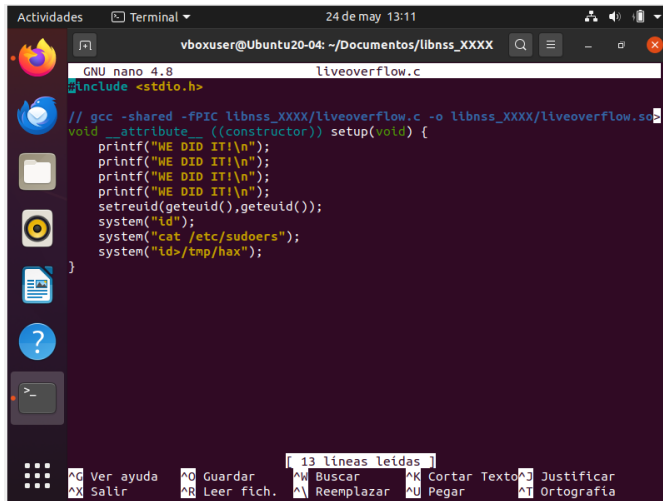
de muchas asignaciones; lo que cambia principalmente son las direcciones bases. Para la realización del ataque, se ocupa que en la ejecución de Sudoedit ciertos espacios de memoria queden en posiciones cercanas y en orden predecible; a esto se le llama heap layout. Los tamaños de los strings del argumento y de las variables de entorno pueden influir en cuáles estructuras quedan después del buffer afectado, así como en los tamaños de estas. Es por esto que se ajustan los argumentos a tamaños específicos, usualmente grandes, que funcionan como desplazamientos relativos para moldear el heap a un layout favorable y repetible; a esto se le llama heap grooming o heap feng shui [27].

El objetivo en este caso es modificar estructuras de NSS para forzar que este cargue la biblioteca especificada `liveoverflow.c`. Este es un objetivo atractivo porque Sudo usa funciones del sistema operativo y esas funciones son administradas por NSS. Específicamente, los strings



```
vboxuser@Ubuntu20-04: ~/Documentos
vboxuser@Ubuntu20-04:~/Documentos$ gcc -shared -fPIC libnss_XXX/liveoverflow.c
-o libnss_XXX/liveoverflow.so.2
libnss_XXX/liveoverflow.c: In function 'setup':
libnss_XXX/liveoverflow.c:9:5: warning: implicit declaration of function 'setreuid' [-Wimplicit-function-declaration]
9 | setreuid(geteuid(),geteuid());
  | ~~~~~^
libnss_XXX/liveoverflow.c:9:14: warning: implicit declaration of function 'geteuid' [-Wimplicit-function-declaration]
9 | setreuid(geteuid(),geteuid());
  |              ^~~~~~
libnss_XXX/liveoverflow.c:10:5: warning: implicit declaration of function 'system' [-Wimplicit-function-declaration]
10 | system("id");
    | ~~~~~^
vboxuser@Ubuntu20-04:~/Documentos$
```

Figura 12. Comando para ejecutar el archivo liveoverflow.c



```
GNU nano 4.8 liveoverflow.c
#include <stdio.h>

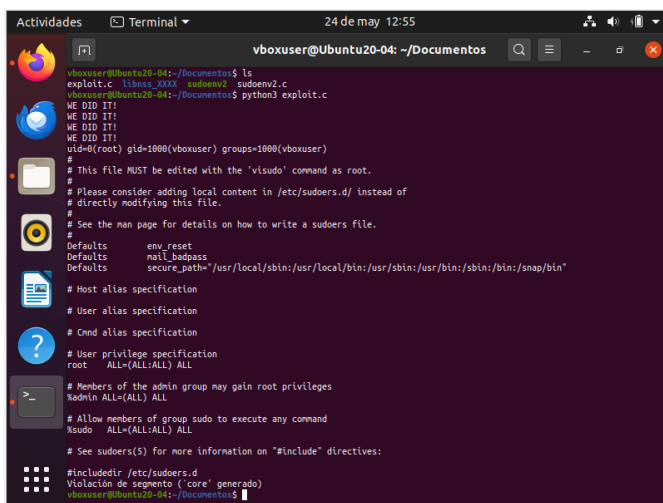
// gcc -shared -fPIC libnss_XXX/liveoverflow.c -o libnss_XXX/liveoverflow.so.2
void __attribute__((constructor)) setup(void) {
    printf("WE DID IT!\n");
    printf("WE DID IT!\n");
    printf("WE DID IT!\n");
    printf("WE DID IT!\n");
    setreuid(geteuid(),geteuid());
    system("id");
    system("cat /etc/sudoers");
    system("id>/tmp/hax");
}
```

Figura 13. Contenido del archivo liveoverflow.c

definidos en `exploit.c` generan un layout del heap predecible en donde ciertos datos del módulo NSS aparecen después del buffer desbordado. Así, el desbordamiento altera la lista `service_library` de objetos internos de glibc llamados `service_user`, el cual es una estructura en memoria para una entrada de servicio de NSS. Esta estructura contiene datos como `char name[0]`, `known` y `library`. Se ocupa sobrescribir `name` con `libnss_XXX/liveoverflow.c`, de esta forma se especifica la ruta relativa de la biblioteca que se ocupa cargar. Más adelante se llama a `__nss_lookup_function` con un `service_user` específico, el puntero `known` se ocupa poner como `NULL` para poder llegar a la llamada de la función `nss_load_library` con el mismo `service_user`. Por último, es necesario poner el puntero `library` a `NULL` para alcanzar la llamada de `dlopen(name)`, que es la función que va a cargar la biblioteca `liveoverflow.c` y, consecuentemente, se ejecuta el ataque de forma exitosa [27]. En resumen, por medio del desbordamiento del búfer se modifican esas tres variables de algún `service_user` a los valores especificados, lo que engaña al programa, así cargando una biblioteca maliciosa.

En la figura 12 se muestra el comando de compilación para el archivo `liveoverflow.c`, el cual tiene que ser en un formato específico para ser cargado en el sistema.

En la figura 13, se muestra el contenido del archivo que se va a ejecutar como una biblioteca encargada de finalizar el ataque aprovechando los privilegios de administrador obtenidos. El constructor que se muestra en la imagen se va a ejecutar en cuanto se llame a la función `dlopen`. Tras varios prints de la frase “WE DID IT!”, se ejecuta `setreuid(geteuid(),geteuid())` que establece el id del usuario real y efectivo del proceso de llamada por el id efectivo actual; la idea es fijar el usuario como root



```
vboxuser@Ubuntu20-04:~/Documentos$ ls
exploit.c libnss_XXX sudoenv2 sudoenv2.c
vboxuser@Ubuntu20-04:~/Documentos$ python3 exploit.c
WE DID IT!
WE DID IT!
WE DID IT!
uid=0(root) gid=1000(vboxuser) groups=1000(vboxuser)
#
# This file MUST be edited with the 'visudo' command as root.
#
# Please consider adding local content in /etc/sudoers.d/ instead of
# directly modifying this file.
# See the man page for details on how to write a sudoers file.
#
Defaults env_reset
Defaults mail_badpass
Defaults secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin"
# Host alias specification
# User alias specification
# Cmnd alias specification
# User privilege specification
root    ALL=(ALL) ALL
# Members of the admin group may gain root privileges
%admin   ALL=(ALL) ALL
# Allow members of group sudo to execute any command
%sudo   ALL=(ALL) ALL
# See sudoers(5) for more information on "#include" directives:
#include_dir /etc/sudoers.d
Violación de segmento ('core' generado)
vboxuser@Ubuntu20-04:~/Documentos$
```

Figura 14. Demostración de la escalación de privilegios mediante CVE-2021-3156

y así el resto de comandos se podrán ejecutar como tal. Seguido se ejecuta `id` que muestra la información del usuario como el nombre, el UID y el GID. También se ejecuta el comando explicado en la sección VI-A. Finalmente, se copia el contenido tras ejecutar `id` en `/tmp/hax`, con el fin de generar evidencia [27].

Finalmente, en la figura 14 se muestra la ejecución del archivo que realiza el ataque por el comando `python3 exploit.c`, que corresponde a la ejecución de un archivo de Python. Se puede observar que el ataque se completa de forma exitosa, por lo que se muestra en pantalla el resultado generado por `liveoverflow.c`. El usuario `vboxuser` obtiene privilegios de administrador y se imprime el contenido del archivo privado por medio del comando `cat /tmp/sudoers`, lo que demuestra la efectividad del ataque. Al final, el programa termina con una violación de

```

969 970      */
970 971      for (to = user_args, av = NewArgv + 1; (from = *av); av++) {
971 972          while (*from) {
972      -      if (from[0] == '\\' && !isspace((unsigned char)from[1]))
973      +      if (from[0] == '\\' && from[1] != '\0' &&
974      +          !isspace((unsigned char)from[1])) {
973 975              from++;
976      +      }
977      +      if (size - (to - user_args) < 1) {
978      +          sudo_warnx(U_("internal error, %s overflow"),
979      +              __func__);
980      +          debug_return_int(NOT_FOUND_ERROR);
981      +      }

```

Figura 15. Cambio del código en una de las partes que era vulnerable

```

262      /* First, check to see if we were invoked as "sudoedit". */
263      proglen = strlen(progname);
264      if (proglen > 4 && strcmp(progname + proglen - 4, "edit") == 0) {
265          progname = "sudoedit";
266          mode = MODE_EDIT;
267          sudo_settings[ARG_SUDOEDIT].value = "true";
268      +      valid_flags = EDIT_VALID_FLAGS;
269      }

```

Figura 16. Cambio del código en la parte vulnerable de sudoedit

segmento debido a la corrupción de partes del heap.

VII. REMEDIACIÓN

VII-A. Propuesta teórica

La recomendación oficial es actualizar lo antes posible a la versión parcheada de Sudo 1.9.5p2 o superior. La versión parcheada corrige la vulnerabilidad que ocasionaba el desbordamiento del búfer, añade mejores verificaciones y arregla el manejo de banderas que funcionaba de manera inconsistente en las versiones pasadas.

Al analizar las diferencias entre una de las versiones afectadas 1.9.5p1 y la versión parcheada 1.9.5p2 en el repositorio Sudo-project, en la figura 15 se puede observar un cambio importante: se añade la verificación `from[1] != '\0'` en el lugar donde ocurría el desbordamiento del búfer, lo que evita realizar lo descrito en la sección V-B.

Por otro lado, en la figura 16 se corrige la inconsistencia en el manejo de banderas que ocurría al usar Sudoedit y que permitía saltarse el escapado de caracteres. De esta forma, al ya no poder hacer el desbordamiento del búfer, no se puede realizar el ataque descrito en la sección V-B.

Para sistemas en donde no es posible parchear inmediatamente, se pueden implementar mitigaciones temporales como deshabilitar en el sistema la funcionalidad Sudoedit o prevenir la ejecución con argumentos maliciosos, restringir el acceso a cuentas locales a solo usuarios confiables y monitorear los registros y el uso de comandos para detectar actividad extraña [28].

A pesar de que este ataque ya no representa un riesgo en las nuevas versiones de Sudo, es importante monitorear los sistemas para verificar que no usen una versión anterior a 1.9.5p1, debido a que esto representa un gran riesgo de seguridad y puede comprometer la información y el funcionamiento del sistema.

```

vboxuser@Ubuntu20-04:~$ sudo nano
[sudo] contraseña para vboxuser:
vboxuser@Ubuntu20-04:~$ sudoedit -s '1234567890\'
malloc(): invalid next size (unsorted)
Abortado ('core' generado)
vboxuser@Ubuntu20-04:~$ sudo --version
Sudo versión 1.8.31
versión del complemento de políticas de sudoers 1.8.31
versión de gramática del archivo Sudoers 46
Sudoers I/O plugin version 1.8.31
vboxuser@Ubuntu20-04:~$

```

Figura 17. Demostración del comportamiento esperado

Respecto a las buenas prácticas de codificación, de este ataque se puede aprender que no siempre hay que confiar en las asunciones o verificaciones realizadas anteriormente en el código, como en este caso asumir que se realizaba el proceso de escapar caracteres, debido a que pueden existir errores que hagan que estas no se cumplan. Es recomendable, en cambio, verificar las condiciones y los diferentes casos que pueden comprometer al programa. También reforzar la revisión del código por diferentes personas para detectar inconsistencias y errores, principalmente en programas críticos como es el caso de Sudo. Relacionado con los principios de seguridad desde el diseño (Security by Design), se recomienda realizar una revisión de diseño segura para identificar posibles vulnerabilidades desde una etapa de desarrollo temprana, cambiando el enfoque de 'Cómo el producto debe funcionar' a 'Cómo el producto puede ser vulnerado' y realizar pruebas y análisis de la seguridad del código [29].

VII-B. Verificación práctica

VII-B1. Configuraciones adicionales: Antes de iniciar, con la demostración es necesario realizar una serie de pasos adicionales. Como se puede observar en la figura 17, tras la ejecución del comando `sudo nano` y `sudoedit -s '1234567890\'` se sabe que no se tienen privilegios de sudo y la vulnerabilidad sigue presente. Además, se ejecuta `sudo --version` para hacer constancia de la versión descargada en el sistema. Con el fin de descargar la versión 1.9.5p2 se requieren los privilegios sudo. Al momento de la descarga de Ubuntu 20.04 en la máquina virtual, no se solicitaba información de la configuración de sudo. Resulta que inicialmente sí se configura el root y se asocia la contraseña del usuario (en esta ocasión vboxuser) a la contraseña de root para mantenimiento, mas no se establece ninguna relación directa con sudo.

Esto último permite entrar al modo recovery de Ubuntu. Para llegar a este estado, lo primero es reiniciar la máquina virtual y al momento del boot presionar ESC, esto redirige

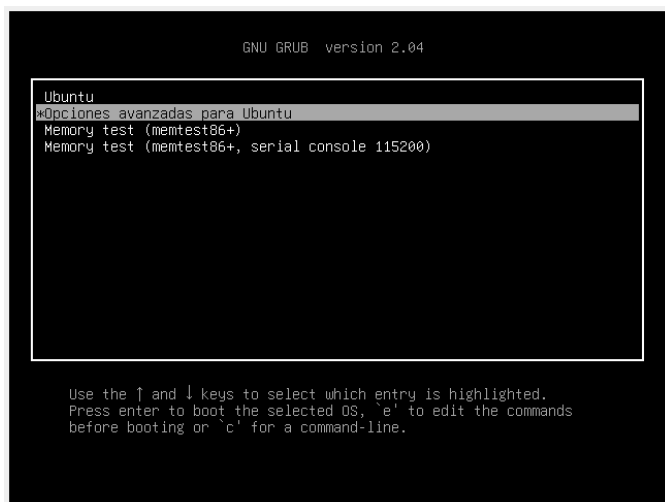


Figura 18. Pantalla de Boot

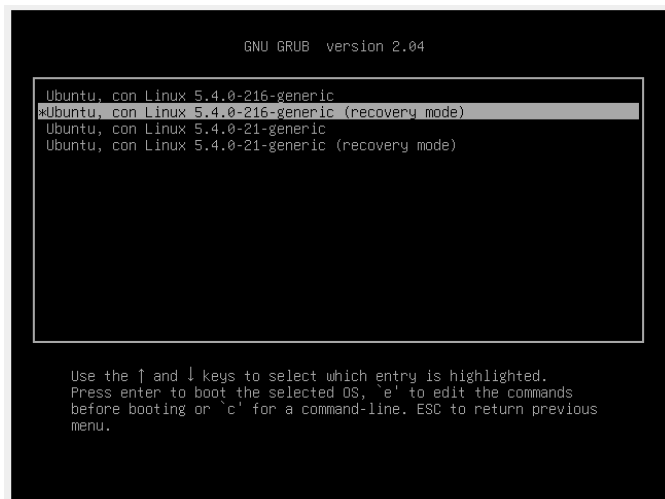


Figura 19. Menú de los Ubuntu disponibles

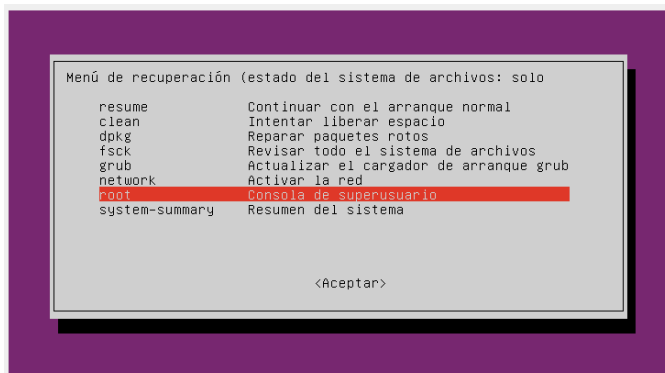


Figura 20. Menú de Ubuntu Recovery

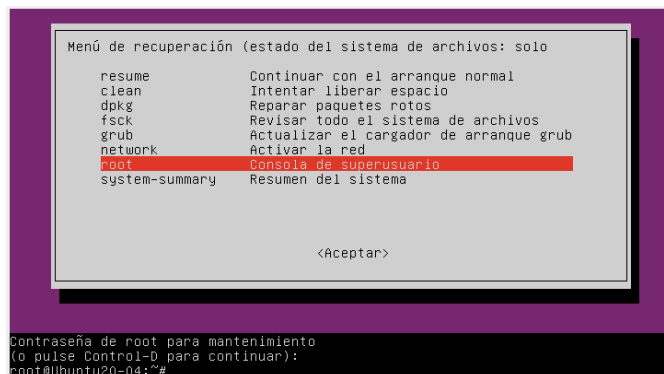


Figura 21. Terminal con permisos root

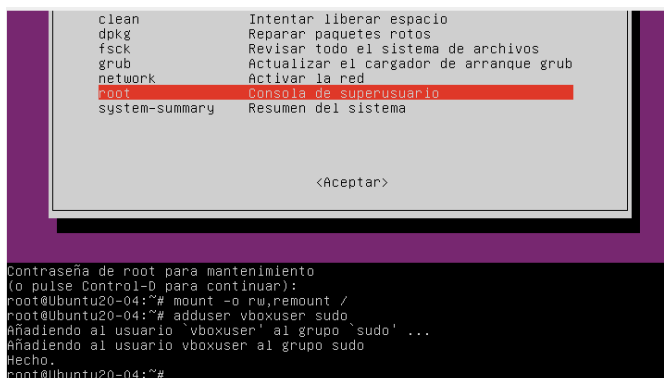


Figura 22. Comandos para colocar vboxuser en sudoers

a la pantalla de la figura 18. En esta figura se debe seleccionar Opciones avanzadas para Ubuntu. Tras esto, se desplegará el menú de las opciones de Ubuntu disponibles, figura 19, y al seleccionar Ubuntu, con Linux 5.4.0-216-generic (recovery mode) se mostrará la pantalla de la figura 20. Desde el modo recovery de Ubuntu es posible escribir el usuario vboxuser como sudoer.

La forma de conseguirlo es accediendo a la terminal de root, seleccionando root. Una vez seleccionada se despliega una terminal como la mostrada en la figura 21. Allí se solicitará la contraseña de mantenimiento de root, y como antes se menciono es la misma que la del usuario vboxuser. Además, como se muestra en la figura 21, se accede como root.

Primero, como se observa en la figura 22, se debe ejecutar `mount -o rw,remount /` para montar el sistema de archivo para poder modificarlo, es necesario ya que recovery mode monta los discos en modo lectura por seguridad. Seguido se ejecuta `adduser vboxuser sudo` para agregar el usuario a sudo. Finalmente se ejecuta `shutdown -r now` para reiniciar el equipo e ingresar con el mismo usuario.

VII-B2. Demostración: Una vez ejecutada, la configuración anterior, se debe descargar la versión 1.9.5p2 de sudo. Por lo tanto, primeros se debe descargar el archivo con sudo con el comando `wget https://www.sudo.ws/dist/sudo-1.9.5p2.tar.gz`. Seguido se debe descomprimir

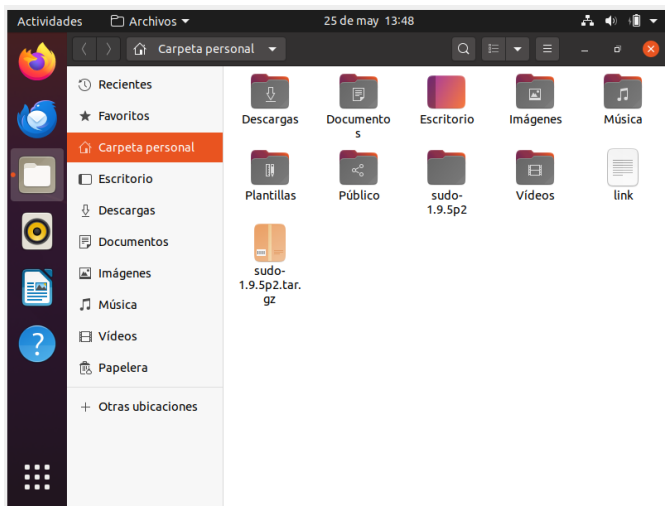


Figura 23. Interfaz con los archivos de sudo 1.9.5p2

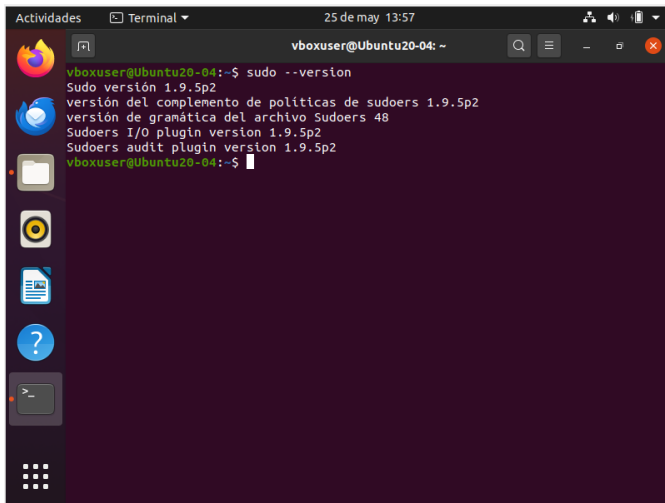


Figura 24. Demostración de la correcta instalación de sudo 1.9.5p2

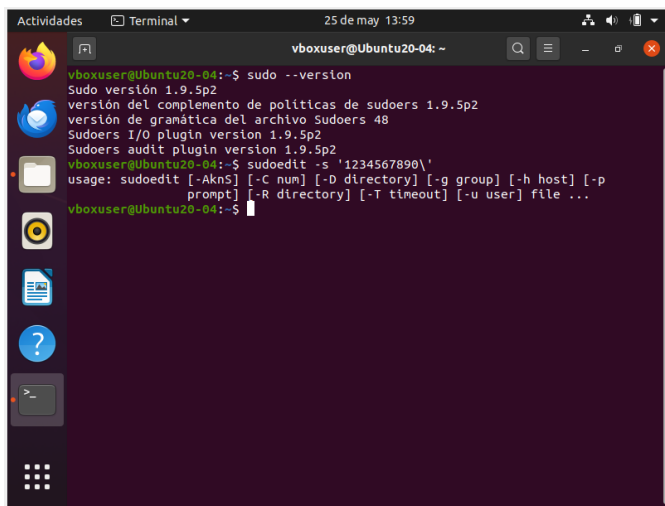


Figura 25. Prueba de la solución

con el comando `tar -xzf sudo-1.9.5p2.tar.gz`, dando como resultado la figura 23. Seguido, dentro de la carpeta, se ejecuta `./configure`, `make` y `sudo make install` para cargar la configuración, compilar sudo e instalar sudo localmente, respectivamente. En la figura 24 se muestra que tras los comandos se instaló correctamente sudo y en la figura 25 que la versión de sudo 1.9.5p2 no se ve afectada por el desbordamiento de heap.

VIII. DISCUSIÓN

En este trabajo se puede analizar cómo un error simple y sutil, en un programa importante como Sudo, el cual ocasiona el desbordamiento de un búfer, puede comprometer la seguridad del sistema al permitir la escalación de privilegios hasta obtener privilegios de administrador. No obstante, a pesar de ser un error simple, pasó muchos años sin ser descubierto y es que realizar un ataque por medio de ese error no es tarea sencilla; manipular el heap para modificar el comportamiento del programa requiere bastante conocimiento técnico y un proceso exhaustivo de análisis, prueba y error que no siempre lleva al resultado deseado. Esto lleva a pensar que pueden existir muchas más vulnerabilidades que no se han descubierto todavía en los sistemas que se usan a diario.

En un mundo en el que volver a lo analógico ya no es una opción, cada día aparecen nuevas actualizaciones o servicios alrededor del mundo. Producto de esto, se da origen a diario a miles de vulnerabilidades. En este contexto, el trabajo de las personas en ciberseguridad es crucial; por ejemplo, en este trabajo se observó que gracias al equipo de exploración de Qualys se logró evitar muchos problemas. Lastimosamente, las personas malintencionadas también cooperan entre sí y son igual de capaces que estos equipos de exploración, por lo cual nunca van a desaparecer los ataques en el mundo.

IX. CONCLUSIONES

En conclusión, en este trabajo se observó la magnitud de los daños que puede ocasionar una vulnerabilidad en Sudo como lo es CVE-2021-3156, considerando desde la escritura o modificación de archivos hasta el acceso a root. Este problema nace en julio de 2011 en el commit 8255ed69, afectando a las versiones anteriores a 1.9.5p2 y fue hasta el 13 de enero del 2021 que se descubrió. Lo que demuestra que vulnerabilidades graves pueden pasar muchos años sin ser descubiertas. Afortunadamente, este ataque no tuvo un impacto considerable, tomando en cuenta su gravedad, y fue parchado unos días después.

Por otro lado, se aprendió que en una vulnerabilidad de tipo desbordamiento, lo más complicado es entender bajo qué condiciones se puede forzar un desbordamiento y cómo se puede redirigir el flujo de ejecución a donde se desee, debido a medidas como ASLR. Como se pudo observar en este trabajo, no se requiere de gran habilidad para realizar el desbordamiento en sí, pero la manipulación del heap es una tarea compleja.

Se puede afirmar que se comprendió y replicó satisfactoriamente la escalación de privilegios producto de la vul-

nerabilidad CVE-2021-3156, mediante la creación de una máquina virtual con la versión de Ubuntu 20.04 vulnerable y la ejecución de un script que logró obtener privilegios de administrador y acceder a un archivo no autorizado; de la misma forma, se verificó que el parche propuesto por los desarrolladores de sudo elimina la vulnerabilidad.

Se recomienda a las empresas verificar que todos sus servicios presenten una versión de sudo mayor o igual a la 1.9.5p2. Además, aplicar el principio de mínimo privilegio, reduciendo el acceso a shell innecesario, que los procesos corran con únicamente los permisos cruciales y evitar en la medida de lo posible el uso de sudo. Como ya se mencionó, hoy en día resulta imprescindible tener un equipo especializado en ciberseguridad, capaz de descubrir vulnerabilidades en los sistemas de la empresa. Asimismo, es recomendable estar monitoreando los logs o actividades anómalas, ya que podrían ser indicios de algún atacante que está investigando los posibles vectores de ataque en el sistema. Otra recomendación es segmentar entornos críticos con el fin de que, en una situación de escalación de privilegios, no se comprometan todos los servicios críticos. Finalmente, es importante tener personal capacitado con el fin de minimizar la introducción de errores en el código.

X. ACRÓNIMOS

CVSS Common Vulnerability Scoring System

CVE Common Vulnerability and Exposures

CAPEC Common Attack Pattern Enumeration and Classification

NIST National Institute of Standards and Technology

CWE Common Weakness Enumeration

PoC Proof of Concept

SAGE Semi-Automatic Ground Environment

NSS Name Service Switch

ASLR Address Space Layout Randomization

XI. REFERENCIAS

- [1] A. Corletti. (2025, Sep.) Qué es mitre. DarFe. Accedido: 2026-05-23. [Online]. Available: <https://darfe.es/hacking-etico/que-es-mitre/>
- [2] Ciberseguridad.com. (2026) ¿qué es cve? explicación de las vulnerabilidades y exposiciones comunes. Ciberseguridad.com. Accedido: 2026-05-23. [Online]. Available: <https://ciberseguridad.com/herramientas/marco-mitre-att-ck/cve-vulnerabilidades-exposiciones-comunes/>
- [3] Wikipedia contributors. (2026) Common vulnerabilities and exposures. Wikipedia, La enciclopedia libre. Wikipedia en español, consultado el 2026-05-23. [Online]. Available: https://es.wikipedia.org/wiki/Common_Vulnerabilities_and_Exposures
- [4] Ciphersafety. (2026) Cwe (common weakness enumeration). Ciphersafety. Accedido: 2026-05-23. [Online]. Available: <https://ciphersafety.com/diccionario/cwe-common-weakness-enumeration/>
- [5] MITRE Corporation. (2026) Cwe: Common weakness enumeration. MITRE. Accedido: 2026-05-23. [Online]. Available: <https://cwe.mitre.org/>
- [6] —. (2026) Capec - common attack pattern enumerations and classifications. MITRE. Accedido: 2026-05-23. [Online]. Available: <https://capec.mitre.org/>
- [7] Wikifrancaise contributors. (2026) Common attack pattern enumeration and classification. FrWiki. Consultado el 2026-05-23. [Online]. Available: https://es.frwiki.wiki/wiki/Common_Attack_Pattern_Enumeration_and_Classification
- [8] Tenable. (2026) Sistema de puntuación de vulnerabilidades comunes (cvss). Tenable. Accedido: 2026-05-23. [Online]. Available: <https://es-la.tenable.com/cybersecurity-guide/principles/common-vulnerability-scoring-system-cvss>
- [9] Wikipedia contributors. (2026) Sudo. Wikipedia, La enciclopedia libre. Wikipedia en español, consultado el 2026-05-23. [Online]. Available: <https://es.wikipedia.org/wiki/Sudo>
- [10] The Linux Foundation. (2022) Classic sysadmin: Configuring the linux sudoers file. The Linux Foundation. [Online]. Available: <https://www.linuxfoundation.org/blog/blog/classic-sysadmin-configuring-the-linux-sudoers-file>
- [11] S. Zivanov. (2024) Cómo utilizar el comando sudo de linux. phoenixNAP. [Online]. Available: <https://phoenixnap.com/kb/linux-sudo>
- [12] H. Arora. How to let users securely edit files using sudoedit on linux. HowtoForge. [Online]. Available: <https://www.howtoforge.com/tutorial/how-to-let-users-securely-edit-files-using-sudoedit/>
- [13] LinuxVox.com. (2026) Understanding and utilizing name service switch (nss) in linux. LinuxVox.com. [Online]. Available: <https://linuxvox.com/blog/nss-in-linux/>
- [14] J. Romero. (2021) ¿qué es el buffer y para qué sirve? [Online]. Available: <https://www.geeknetic.es/Buffer/que-es-y-para-que-sirve>
- [15] Fortinet. Desbordamiento de búfer. Fortinet. [Online]. Available: <https://www.fortinet.com/lat/resources/cyberglossary/buffer-overflow>
- [16] MITRE Corporation, “Cwe-193: Off-by-one error,” Common Weakness Enumeration (CWE), 2006, versión de la página: 2026-04-30. Abstraction: Base. [Online]. Available: <https://cwe.mitre.org/data/definitions/193.html>
- [17] —, “Cwe-170: Improper null termination,” Common Weakness Enumeration (CWE), 2006. [Online]. Available: <https://cwe.mitre.org/data/definitions/170.html>
- [18] —, “Cwe-122: Heap-based buffer overflow,” MITRE Corporation, 2006. [Online]. Available: <https://cwe.mitre.org/data/definitions/122.html>
- [19] Cero Uno. (2016) ¿qué es un heap overflow? Cero Uno. [Online]. Available: <https://cerounosoftware.com.mx/2016/08/02/qu%C3%A9-es-un-heap-overflow/>
- [20] Wikipedia contributors. (2026) Address space layout randomization. Wikipedia, La enciclopedia libre. [Online]. Available: https://en.wikipedia.org/wiki/Address_space_layout_randomization
- [21] Black Duck. Fuzz testing. Black Duck. [Online]. Available: <https://www.blackduck.com/glossary/what-is-fuzz-testing.html>
- [22] “Cve-2021-3156: Sudo heap-based buffer overflow vulnerability,” <https://nvd.nist.gov/vuln/detail/CVE-2021-3156>, 2021, accessed: 2026-05-20.
- [23] Sudo Project, “sudo: Utility to execute a command as another user,” GitHub repository, 2025, Último commit: May 22, 2026. Versión más reciente: 1.9.17p2 (Jul 26, 2025). [Online]. Available: <https://github.com/sudo-project/sudo>
- [24] Qualys Research Team, “Cve-2021-3156: Heap-based buffer overflow in sudo (baron samedit),” <https://blog.qualys.com/vulnerabilities-threat-research/2021/01/26/cve-2021-3156-heap-based-buffer-overflow-in-sudo-baron-samedit>, 2021, qualys vulnerability analysis and disclosure write-up for CVE-2021-3156. Accessed: 2026-05-20.
- [25] Q. S. Advisory, “Baron samedit: Heap-based buffer overflow in sudo (cve-2021-3156),” Qualys, Security Advisory, 01 2021, consultado: 2026-05-21. [Online]. Available: <https://www.qualys.com/2021/01/26/cve-2021-3156/baron-samedit-heap-based-overflow-sudo.txt>
- [26] MITRE Corporation, “Capec-100: Buffer overflow,” Common Attack Pattern Enumeration and Classification (CAPEC), 2017, abstraction: Standard. Última modificación: 2021-10-21 (versión 3.6). [Online]. Available: <https://capec.mitre.org/data/definitions/100.html>
- [27] LiveOverflow, “Sudo vulnerability walkthrough,” <https://www.youtube.com/playlist?list=PLhixgUqwRTjy0gMuT4C3bmjeZjuNQyqdx>, accessed: 2026-05-22.
- [28] SentinelOne. (2026) CVE-2021-3156: Sudo Privilege Escalation Vulnerability. SentinelOne. [Online]. Available: <https://www.sentinelone.com/vulnerability-database/cve-2021-3156/>
- [29] Microsoft. Security Development Lifecycle (SDL) Practices. Microsoft. [Online]. Available: <https://www.microsoft.com/en-us/securityengineering/sdl/practices>